



**FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY
AND RESEARCH**

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y. 2026-27 Semester-3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms



PRACTICAL-2

Practical AIM:

To implement linear and binary search using both iterative and recursive approaches.

2.1

Problem Definition:

A security guard at a parking lot checks vehicles one by one from the entrance to find a car with a specific license plate. Sometimes he starts from the entrance, sometimes he calls a helper who starts from where the guard left off. Given a list of license plates and a target plate, implement both approaches — one that checks plates one by one from the start, and one where the function calls itself to continue checking — and report the position of the target plate if found.

Describe the approach you used. What happens in your solution if the target plate appears more than once in the list? Does it find the first occurrence, the last, or just any one?

Output:

```
Enter no of licence plates:3
Enter licence plates no:4567
3213
4569
Enter target plate:4567

Licence plate found1
```

Difficulty Faced:

The main difficulty was implementing both iterative and recursive approaches to search for the target license plate. Handling the stopping condition in recursion was also important to avoid infinite calls.

Solution:

The iterative approach checks each plate from the beginning using a loop, while the recursive approach checks one plate and calls itself for the next position. Both approaches return the first occurrence of the target plate because they search from the beginning and stop as soon as the target is found.

Outcome:

Successfully implemented both iterative (linear search) and recursive search approaches. Both methods return the position of the first occurrence of the target plate. If the target is not present, the program indicates that it was not found.

2.2

Problem Definition:

A librarian maintains a sorted catalog of book codes and needs to locate a specific code quickly. Instead of going through every book, she opens the catalog to the middle, checks whether the target is to the left or right, and repeats. Given a sorted list of book codes and a target code, implement both approaches — one using a loop and one where the function calls itself — and report the position of the target code.

Describe the approach you used. What is the one condition your input must satisfy for this approach to work correctly? What would go wrong if that condition was not met?

Output:

```
Enter number of book codes:3
Enter book codes:345
654
643
Enter target book code:345

Book code found at position1
Book code found at position1
```

Difficulties Faced:

The main difficulty was understanding how to reduce the search area by checking the middle element. In the recursive method, defining the correct base condition was also important.

Solution:

Used binary search in both iterative and recursive approaches. The middle element is compared with the target, and the search continues only in the left or right half. The required condition is that the list must be sorted. If the list is not sorted, the search may move to the wrong half and fail to find the target.

Outcome:

Successfully implemented iterative and recursive binary search to locate the target book code efficiently.

The most important condition is that the input list must be sorted. Binary search depends on the ordering of elements to decide which half can be eliminated. If the list is not sorted, the program may

discard the half containing the target and therefore return an incorrect result or fail to find an existing target.

Key Questions:

1.[Problem 1] If the target plate appears near the end of a very long list, your solution still checks every plate before it. Can you think of any property the input would need to have for you to skip checking some plates entirely? What does Problem 2.2 tell you about this?

Sol: The list needs to be sorted so that we can skip half of the elements using binary search. Problem 2.2 tells us that binary search works correctly only on a sorted list.

2. [Problem 2] What is the consequence of applying your solution to a list that is not sorted? Would it always give a wrong answer, or only sometimes? Explain your reasoning without running any example.

Sol: If the list is not sorted, binary search may give the wrong answer because it can discard the half containing the target. It will not always be wrong; sometimes it may still find the target by chance.

Github Link:

<https://github.com/brindajivani/DSA/tree/main/Practical2>